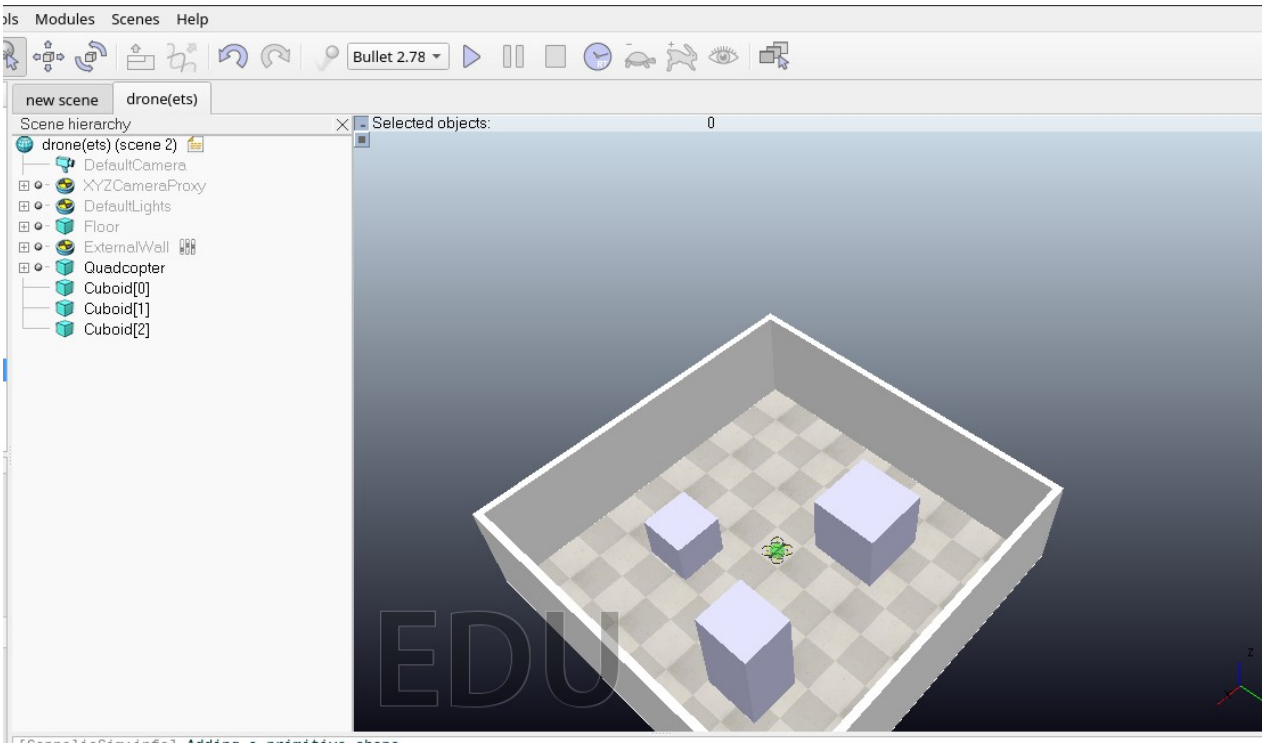
LAPORAN TEKNIS UJIAN TENGAH SEMESTER

PEMETAAN LINGKUNGAN 3D MENGGUNAKAN QUADCOPTER

Akuisisi depth · Point cloud 3D · Voxelisasi · Color mapping ketinggian



Gambar 1. Scene pengujian CoppeliaSim: Quadcopter, Floor, ExternalWall dan tiga Cuboid.

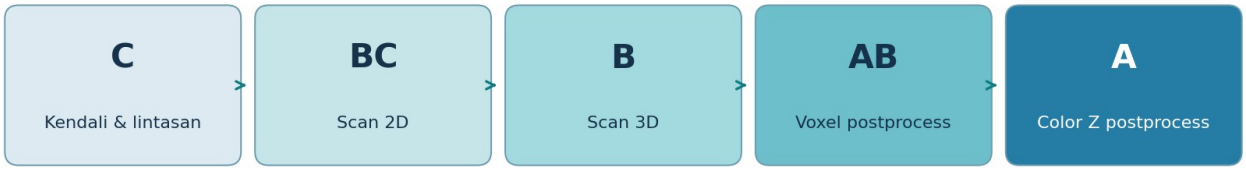
Ringkasan misi

Simulasi mengendalikan Quadcopter melalui waypoint FlightMission Lua, merekam data kedalaman sensor, membentuk point cloud dalam koordinat dunia, dan mengolah hasilnya menjadi voxel terisi. Visualisasi akhir menampilkan muka voxel 3D dengan sumbu Cartesian X/Y/Z numerik, pewarnaan sesuai elevasi Z, serta lintasan penerbangan.

Perangkat utama	Realisasi teknis
Simulator dan aset	CoppeliaSim; Quadcopter; Fast 3D Laser Scanner; Floor; ExternalWall; Cuboid[0]–[2].
Pengendalian	FlightMission Lua + target dummy + pengendali Quadcopter bawaan.
Pemrosesan	Python; ZeroMQ Remote API; NumPy; Matplotlib.
Sasaran rubrik	Grade C → BC → B → AB → A secara kumulatif.

1. Ketentuan UTS dan pemetaan capaian

Penilaian bersifat kumulatif: pencapaian suatu grade mensyaratkan seluruh tahap pada grade di bawahnya. Penambahan fitur orisinal atau kompleksitas dapat menjadi penilaian tambahan sesuai ketentuan UTS.



Gambar 2. Urutan capaian UTS yang berjenjang dari kontrol hingga color mapping ketinggian.

Grade	Kriteria tugas	Realisasi / bukti yang diperlukan
C	Kontrol drone dan tampilkan jalur	Waypoint Lua; posisi Quadcopter dicatat; lintasan biru dalam grafik.
BC	Pindai lingkungan dalam 2D	Proyeksi titik (X,Y) dan lintasan pada 01_mapping_2d.png.
B	Lakukan pemindaian 3D	Depth sensor → XYZ dunia; ekspor raw_scan.ply dan data NPZ.
AB	Kurangi titik menjadi voxel (postprocess)	Pengelompokan XYZ pada kisi 3D; keluaran voxel_xyz.csv dan PLY.
A	Color mapping ketinggian (postprocess)	Warna voxel berdasarkan koordinat Z; sumbu Z numerik dan colorbar meter.

Ketentuan nilai tambahan: orisinalitas atau kompleksitas tambahan dapat meningkatkan grade satu tingkat; kemiripan dengan pekerjaan yang telah dikumpulkan dapat menurunkan satu tingkat; pekerjaan tidak lengkap atau tidak sesuai ketentuan memperoleh Grade D.

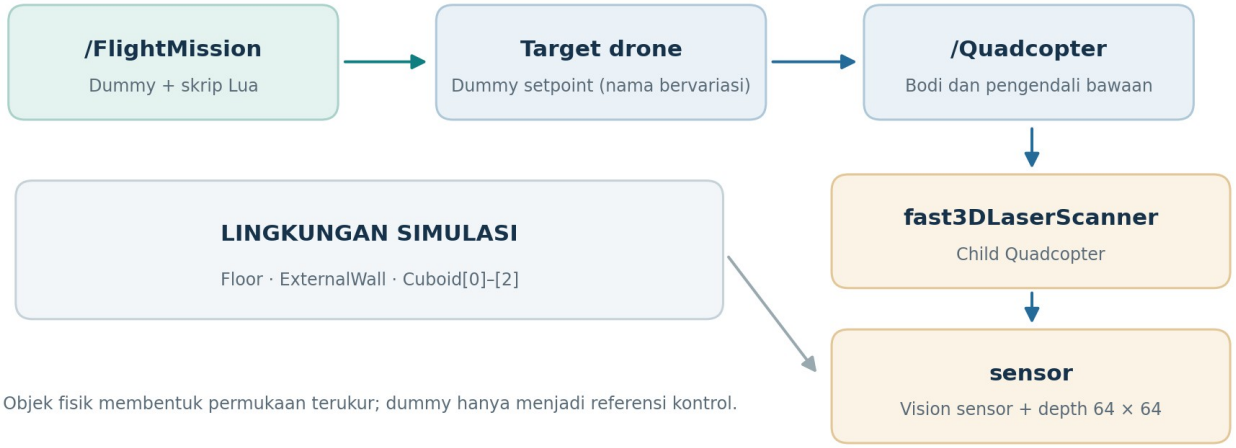
1.1. Syarat pengamatan sensor

Posisi dan orientasi sensor harus memungkinkan pendeteksian permukaan lantai (ground), sisi/badan objek, dan bidang puncak objek. Cakupan diperoleh dari gabungan sudut pandang selama lintasan; permukaan yang tidak terlihat tidak dianggap telah terukur.

1.2. Berkas penyerahan

Berkas / media	Isi
Video proses mapping	Rekaman penerbangan, lintasan dan aktivitas pembacaan sensor.
Video hasil 3D mapping	Rotasi grafik 3D, sumbu X/Y/Z dan perubahan warna menurut Z.
Repositori GitHub	Scene .ttt, program .py, skrip .lua dan laporan .pdf; tautan video disertakan.

2. Aset simulasi dan struktur objek

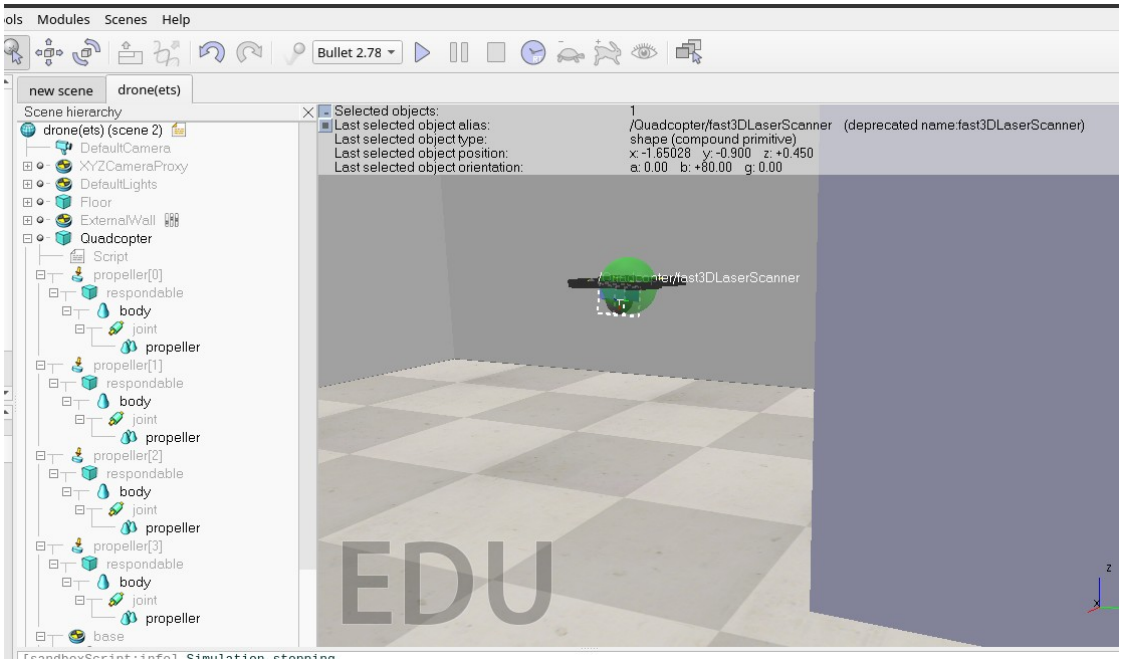


Gambar 3. Hierarki dan hubungan kendali FlightMission, target dummy, Quadcopter, scanner, dan objek lingkungan.

Objek / dummy	Fungsi teknis
/Quadcopter	Bodi drone; referensi pose untuk transformasi titik dan rekaman lintasan.
/FlightMission	Dummy scene-root yang memuat skrip simulasi Lua dan daftar waypoint.
Target drone	Dummy setpoint yang diikuti pengendali bawaan Quadcopter; nama path dapat bervariasi.
fast3DLaserScanner	Sensor yang terpasang sebagai child Quadcopter; membaca keadaan lingkungan.
.../sensor	Vision sensor depth dengan resolusi terbaca 64×64 piksel, near 0,05 m, far 5,00 m.
Floor / ExternalWall	Bidang ground dan batas ruang untuk pembacaan permukaan.
Cuboid[0]–[2]	Tiga objek uji untuk pengamatan sisi, tinggi dan puncak.

2.1. Penempatan sensor

Scanner dipasang pada Quadcopter dengan orientasi terhadap badan yang memungkinkan pengamatan permukaan dari sejumlah posisi terbang. Parameter perspektif dan jarak clipping dibaca dari properti sensor, bukan disisipkan sebagai nilai konstanta rekonstruksi. Cakupan aktual sisi dan puncak Cuboid bergantung pada garis pandang setiap waypoint.



Gambar 4. Konfigurasi sensor pada Quadcopter dalam CoppeliaSim (tangkapan layar scene).

3. Kendali drone dan pemenuhan Grade C

Skrip FlightMission Lua menjalankan urutan waypoint pada dua ketinggian penerbangan. Perpindahan target dummy dilakukan bertahap; drone fisik tetap dikendalikan oleh pengendali bawaan Quadcopter. Garis lintasan dibuat dari posisi dunia pada setiap langkah simulasi.

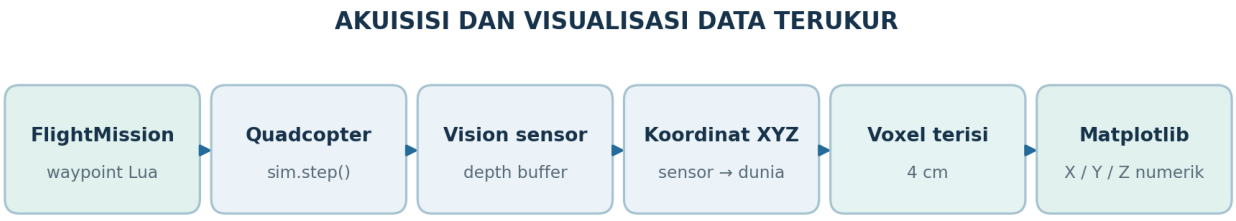
```
local fraction = math.min(1, TARGET_SPEED * dt / len)
sim.setObjectPosition(target, sim.handle_world, {
    setpoint[1] + fraction * dx,
    setpoint[2] + fraction * dy,
    setpoint[3] + fraction * dz
})
signal('droneWaypoint', index)
```

`sim.setObjectPosition()` memindahkan target, bukan langsung mengubah pose bodi Quadcopter. `sim.addDrawingObject()` membuat jejak garis. Sinyal `droneWaypoint` mencatat waypoint aktif, sedangkan `droneMappingDone` menandai akhir misi bagi Python.

3.1. Rekaman lintasan pada Python

```
sim.setStepping(True)
sim.startSimulation()
while True:
    sim.step()
    now = sim.getSimulationTime()
    pos = sim.getObjectPosition(drone, sim.handle_world)
    path.append([now, *pos])
```

Mode stepping menyinkronkan pergerakan simulasi dengan pembacaan pose dan sensor. Setiap baris lintasan berisi waktu simulasi dan koordinat dunia (t, X, Y, Z), sehingga lintasan dapat divisualisasikan ulang tanpa menjalankan simulasi.



Gambar 5. Rangkaian teknis: kendali terbang, data depth, transformasi koordinat, voxelisasi dan grafik Cartesian.

4. Pemindaian 2D dan 3D: Grade BC dan B

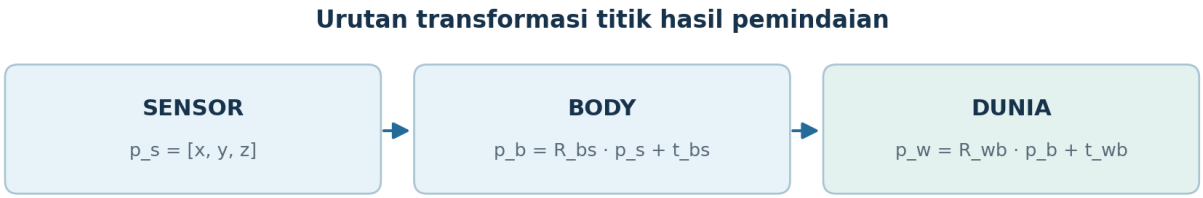
Depth buffer vision sensor berisi kedalaman untuk setiap posisi piksel. Opsi 1 pada pemanggilan API menghasilkan jarak dalam meter. Titik valid disaring berdasarkan near/far clipping dan batas jarak pemindaian agar permukaan badan scanner atau data di luar jangkauan tidak masuk dalam point cloud.

```
raw, resolution = sim.getVisionSensorDepth(sensor_handle, 1)
D = np.frombuffer(raw, dtype=np.float32).reshape(height, width)
valid = np.isfinite(D) & (D > d_min) & (D < d_max)
```

4.1. Rekonstruksi XYZ sensor

$$P_{sensor} = [X_s, Y_s, Z_s]^T = [d \cdot q_x(u), d \cdot q_y(v), d]^T$$

Parameter sudut pandang sensor menentukan fungsi q_x dan q_y pada setiap piksel (u,v). Keluaran awal berada pada frame sensor. Titik diubah ke frame Quadcopter dan frame dunia menggunakan matriks transformasi homogen 4×4.



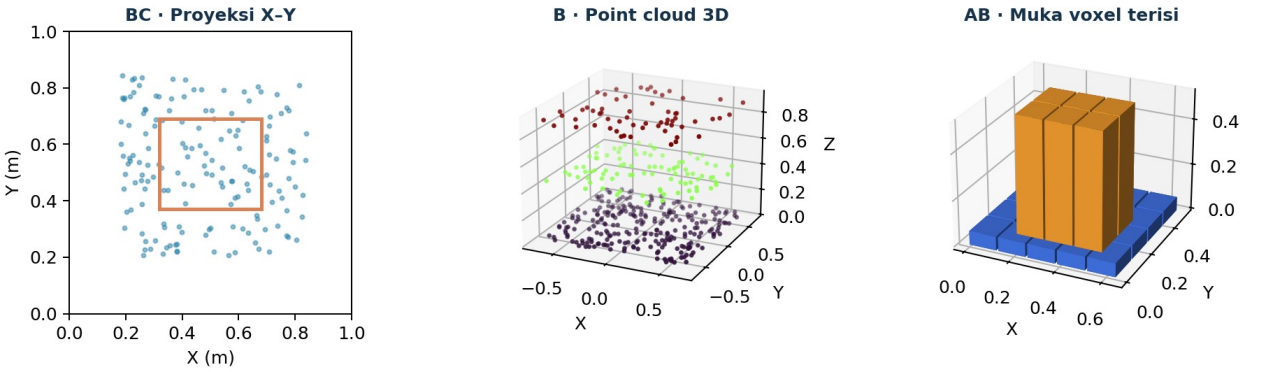
Gambar 6. Transformasi titik dari koordinat sensor ke badan Quadcopter, kemudian ke koordinat dunia.

$$P_{world} = T_{world \leftarrow body} \cdot T_{body \leftarrow sensor} \cdot P_{sensor}$$

Transformasi matriks menggunakan rotasi dan translasi dari `sim.getObjectMatrix()`. Proyeksi 2D diambil dari kolom X–Y hasil pengukuran dunia; model 3D mempertahankan komponen Z. Pemisahan tersebut mendukung Grade BC dan B dari satu sumber point cloud.

5. Implementasi awal dan masalah pemetaan

Versi dasar mengakuisisi depth tiap 0,45 detik waktu simulasi dengan `PIXEL_STRIDE = 6`. Sensor 64×64 menyediakan 4.096 posisi piksel, tetapi hanya 121 posisi yang dicoba pada setiap frame. Titik valid dikelompokkan pada kisi voxel 0,08 m dan satu titik representatif ditampilkan untuk setiap sel.



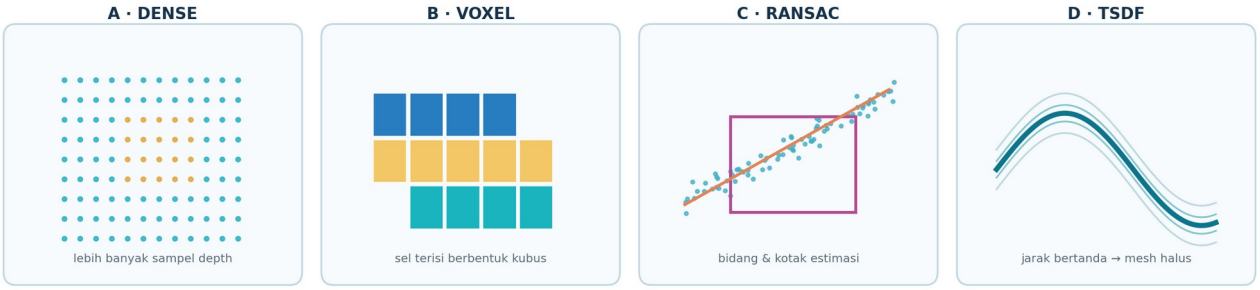
Gambar 7. Perbedaan tahapan keluaran: proyeksi 2D, point cloud 3D, dan representasi muka voxel (ilustrasi).

Aspek awal	Konsekuensi pada hasil
Jarak antar sampel 6 piksel	Jumlah titik yang diukur per frame terbatas; detail kecil mudah terlewat.
Interval 0,45 s	Jarak antar posisi scan lebih jauh pada kecepatan terbang yang sama.
Voxel 0,08 m	Detail kisi lebih kasar; satu sel setara kubus berukuran 8 cm.
Pusat voxel berupa titik	Bidang kotak masih tampak berlubang dan tidak memiliki muka kubus.
Sudut pandang tertentu	Ground, sisi, dan puncak Cuboid belum tentu semua berada dalam jangkauan sensor.

5.1. Fokus pengembangan

Perbaikan ditujukan pada densitas sampel depth, representasi sel voxel yang terisi, dan keterbacaan elevasi 3D. Peningkatan jumlah sampel dan perubahan ukuran grid tidak secara otomatis membuktikan berkurangnya galat geometri; pengujian akurasi memerlukan geometri acuan scene.

6. Dasar teori: empat metode peningkatan peta 3D



Gambar 8. Empat pendekatan: A dense point cloud; B solid voxel; C RANSAC dan rekonstruksi kotak; D TSDF fusion (diagram konseptual).

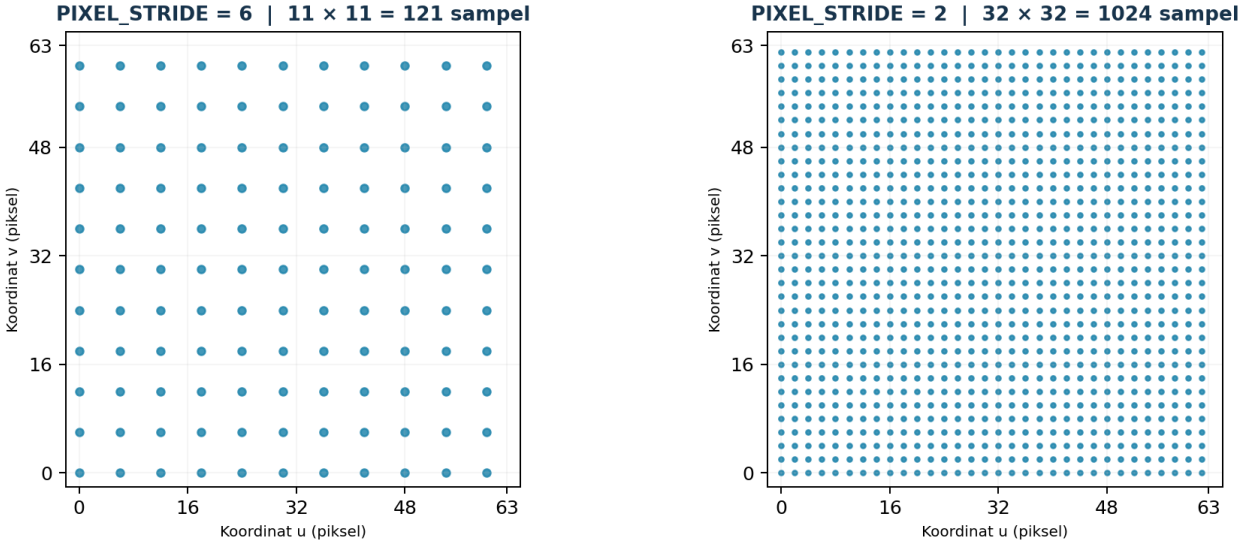
Metode	Prinsip teknis	Keterkaitan voxel	Status proyek
A. Dense point cloud	Menambah pembacaan piksel depth dan jumlah frame sepanjang lintasan.	Opsional; masukan untuk voxelisasi.	Diterapkan di kode final.
B. Solid voxel reconstruction	Memetakan titik ke sel grid 3D terisi; menggambar muka sel yang terlihat.	Eksplisit; representasi voxel langsung.	Diterapkan di kode final.
C. Plane fitting + box	RANSAC untuk bidang datar, dilanjutkan pengelompokan dan estimasi geometri Cuboid.	Opsional; dapat memakai titik hasil voxel.	Dikaji dan diuji dalam versi pengembangan, tidak di kode akhir.
D. TSDF fusion + meshing	Menggabungkan depth multi-frame dalam jarak bertanda dan mengekstraksi permukaan kontinu.	Eksplisit; voxel menyimpan signed distance.	Kajian alternatif; tidak diterapkan.

6.1. Pemilihan metode untuk misi UTS

Tiga metode yang masuk proses pengembangan ialah A, B, dan C. A dan B dipertahankan sebagai pipeline akhir karena keluaran tetap berbasis voxel terukur dan mendukung Grade AB–A. C diuji sebagai pendekatan estimasi bidang/kotak, tetapi hasil final tidak memakai fitting; D tidak dipilih karena fokusnya pembentukan permukaan halus, bukan visualisasi sel voxel yang eksplisit.

Status metode berbeda dengan rubrik nilai: metode A/B/C/D adalah opsi pemrosesan, sedangkan Grade A–C adalah capaian tugas UTS.

7. Metode A: peningkatan densitas point cloud



Gambar 9. Posisi piksel yang dicoba pada depth frame 64×64 ketika stride 6 dan stride 2.

$$N_{pixel/scan} = \text{ceil}(64 / \text{stride}) \times \text{ceil}(64 / \text{stride})$$

Parameter	Versi awal	Versi akhir	Perubahan
PIXEL_STRIDE	6 piksel	2 piksel	3× lebih rapat per sumbu
Sampel kandidat / frame	11 × 11 = 121	32 × 32 = 1.024	1.024 / 121 = 8,46×
ACQUIRE_EVERY	0,45 s	0,20 s	2,25× frekuensi pemindaian
Frame kandidat / detik	2,22 frame/s	5,00 frame/s	2,25×
Sampel kandidat / detik	121 / 0,45 ≈ 269	1.024 / 0,20 = 5.120	≈ 19,04×

Nilai di atas adalah kapasitas sampling ideal per sensor dan per detik simulasi. Jumlah titik valid tetap dipengaruhi kedalaman, clipping, posisi sensor, permukaan yang tampak, dan durasi misi. Throughput nyata belum ditentukan dari hasil pemindaian scene.

```
parser.add_argument('--pixel-stride', type=int, default=2)
parser.add_argument('--acquire-every', type=float, default=.20)
u = np.arange(0, width, stride)
v = np.arange(0, height, stride)
```

`np.arange()` menentukan lokasi piksel yang dibaca; interval simulasi mengatur frekuensi pengambilan frame. Kombinasi keduanya meningkatkan kesempatan observasi permukaan, tanpa mengubah resolusi asli vision sensor 64×64.

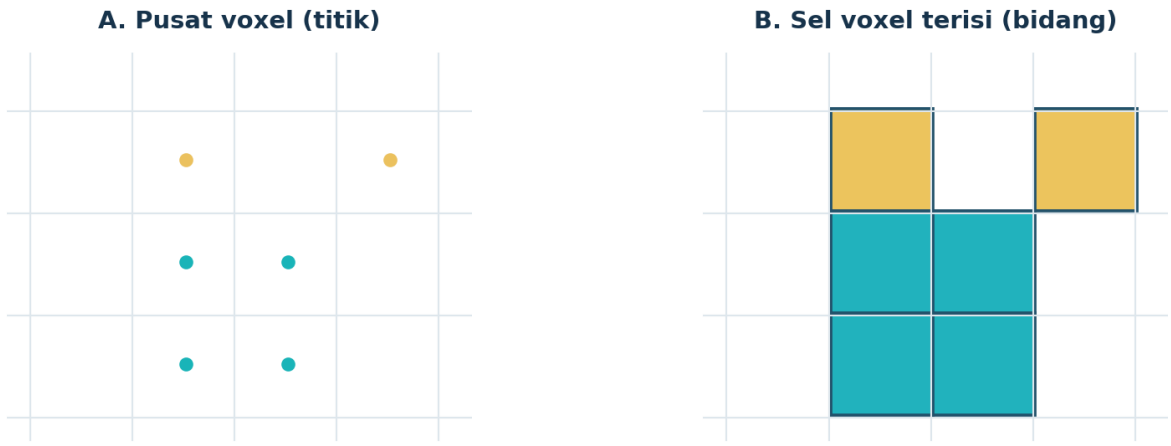
8. Metode B: voxelisasi dan representasi kubus

Voxelisasi mengubah titik (X,Y,Z) menjadi indeks sel pada grid 3D berukuran s. Sel yang menerima sedikitnya satu titik pengukuran ditandai terisi. Sel tidak diisi hanya karena berada di dalam bentuk Cuboid yang diperkirakan.

$$k = \text{floor}(P_{world} / s) \; ; \; P_{center} = (k + 0{,}5) \times s$$

```
keys = np.unique(np.floor(points / size).astype(np.int64), axis=0)
occupied = set(map(tuple, keys))
for key in keys:
    for offset, face in sides:
        if tuple(key + offset) not in occupied:
            quads.append(vertices[list(face)])
```

`np.floor()` membentuk indeks voxel; `np.unique()` menghapus sel duplikat. Pemeriksaan `occupied` menghilangkan muka yang bersebelahan dengan voxel terisi sehingga hanya muka luar yang dirender sebagai `Poly3DCollection`.



Gambar 10. Perbedaan pusat voxel berbentuk titik dengan tampilan muka kubus terisi; ilustrasi representasi.

Ukuran grid	Volume per voxel	Implikasi
0,08 m (awal)	$0{,}08^3 = 0{,}000512\ m^3$	Representasi lebih kasar.
0,04 m (akhir)	$0{,}04^3 = 0{,}000064\ m^3$	Ukuran linier 2× lebih kecil; volume sel 8× lebih kecil.

Dalam volume geometris yang sama, jumlah sel potensial menjadi 8× lebih besar apabila seluruh volume terisi. Jumlah voxel teramati tidak harus meningkat 8× karena sensor hanya mengukur permukaan dan sebagian sel tidak memiliki data. `--display-limit=4500` dapat mengasarkan mesh tampilan; ekspor voxel tetap mengikuti parameter dasar 4 cm.

9. Metode C dan D dalam pengembangan sistem

9.1. C — Plane fitting dan estimasi Cuboid

RANSAC memilih sampel titik secara acak untuk memperkirakan bidang, kemudian menghitung jumlah titik yang cocok dalam ambang jarak tertentu. Setelah lantai/dinding dipisahkan, kelompok titik sisa dapat dipakai sebagai kandidat batas Cuboid. Persamaan bidang umum:

$$aX + bY + cZ + d = 0$$

```
# Contoh metode C (prototipe; tidak ada pada main.py final)
model, inliers = cloud.segment_plane(
    distance_threshold=0.04, ransac_n=3, num_iterations=250)
# Pengelompokan residual dan fitting box adalah tahap lanjutan.
```

Metode C sempat masuk pengembangan rekonstruksi, tetapi gambar kotak hasil estimasi tidak digunakan pada keluaran final. Kotak fitting dapat menunjukkan batas geometri yang diperkirakan, namun tidak membuktikan seluruh enam bidang objek sudah terscan. Pemisahan antara permukaan terukur dan geometri inferensi harus dipertahankan.

9.2. D — TSDF fusion dan mesh permukaan

Truncated Signed Distance Function (TSDF) menyimpan jarak bertanda dari sel volumetrik ke permukaan terdekat. Integrasi depth dari beberapa pose dapat menghasilkan mesh yang lebih kontinu setelah ekstraksi isosurface. Metode ini memerlukan pengelolaan pose sensor, skala voxel dan biaya komputasi lebih tinggi.

Pertimbangan	B. Occupied voxel	D. TSDF

Nilai grid	Status sel terukur/terisi	Jarak bertanda dan bobot integrasi
Representasi akhir	Kubus dan muka voxel	Mesh kontinu / permukaan halus
Keluaran pada proyek	Diterapkan	Tidak diterapkan
Interpretasi ruang kosong	Tidak mengasumsikan interior objek	Dapat menggabungkan bukti free-space sesuai metode

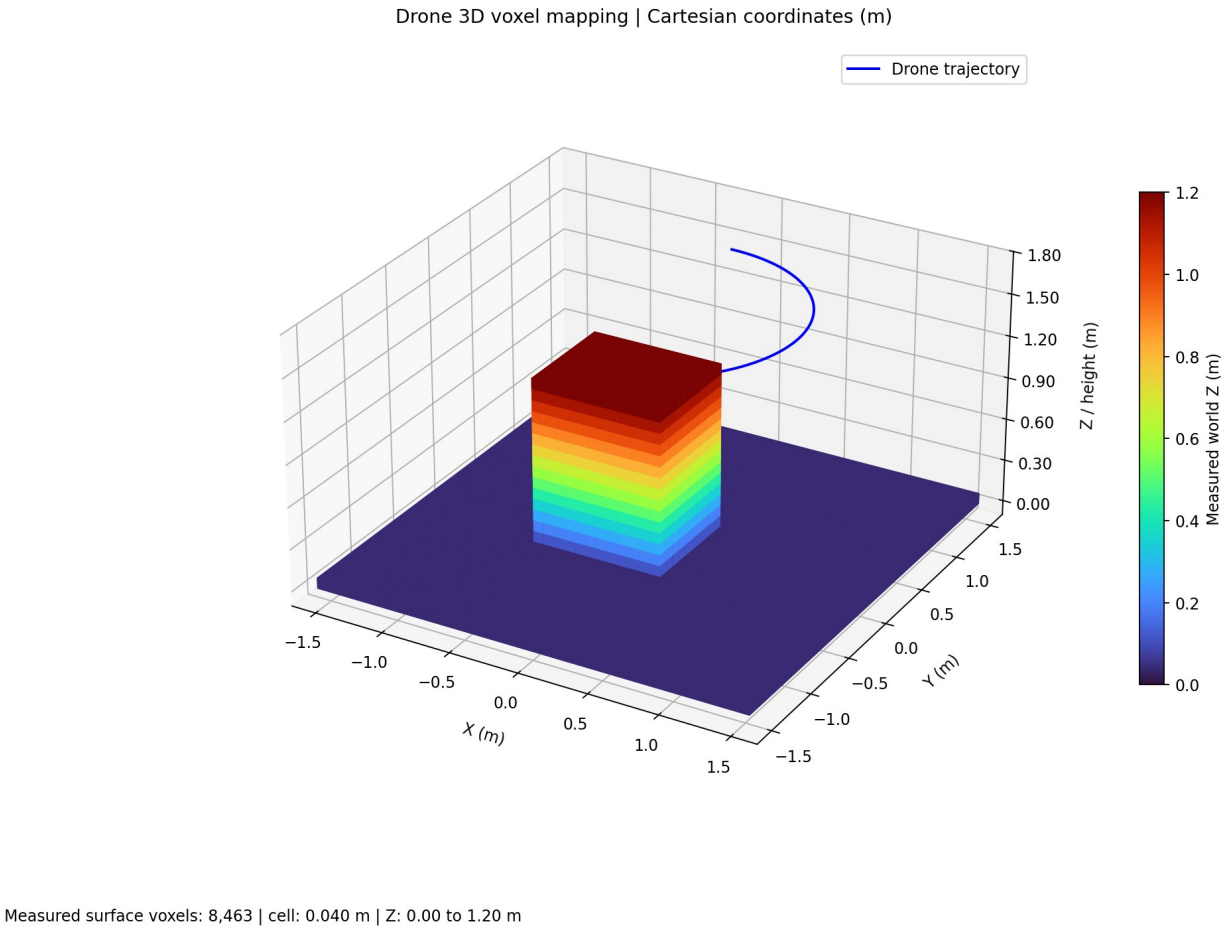
Tiga opsi yang dibahas dalam proses pengembangan ialah A+B+C; hanya A+B tersimpan pada implementasi final. D menjadi pembanding teoretis, bukan fitur yang telah diuji pada scene.

10. Postprocess warna elevasi dan pemenuhan Grade A

Nilai Z untuk setiap voxel berasal dari koordinat dunia. Fungsi colormap mengubah elevasi rendah menjadi warna biru dan elevasi lebih tinggi menjadi hijau–kuning–merah. Sumbu Cartesian menampilkan angka X, Y dan Z dalam meter; colorbar menjelaskan hubungan warna dengan elevasi dunia.

$$t = (Z - Z_{min}) / \max(Z_{max} - Z_{min}, \varepsilon) \; ; \; RGB = turbo(t)$$

```
mesh = Poly3DCollection(quads, facecolors=face_colors, linewidths=0)
ax.set_xlabel('X (m)')
ax.set_ylabel('Y (m)')
ax.set_zlabel('Z / height (m)')
ax.zaxis.set_major_formatter(ticker.FormatStrFormatter('%.2f'))
fig.colorbar(cm.ScalarMappable(norm=norm, cmap=cmap), ax=ax)
```



Gambar 11. Grafik voxel Cartesian dengan nilai sumbu X/Y/Z, gradasi warna Z, dan lintasan biru. Ilustrasi uji offline sintetis; bukan hasil scan scene.

Batas interpretasi: warna menunjukkan elevasi Z absolut voxel, bukan estimasi tinggi fisik setiap Cuboid. Tinggi relatif terhadap lantai baru dihitung setelah koordinat dasar referensi ditetapkan. Grade A ditunjukkan oleh postprocess pemetaan warna ketinggian, bukan rekonstruksi sisi objek yang tidak terukur.

11. Evaluasi kuantitatif dan rancangan pengujian

Perbaikan parameter dapat dihitung tanpa menjalankan eksperimen, tetapi peningkatan kualitas geometri memerlukan pembandingan terhadap dimensi dan pose Cuboid pada scene. Data pengukuran nyata harus dipisahkan dari contoh hitungan dan hasil sintetis.



Gambar 12. Alur evaluasi akurasi: geometri ground truth scene, point cloud/voxel hasil sensor, dan metrik kuantitatif.

Metrik	Definisi / rumus	Satuan
Galat dimensi	$\lvert L_{est} - L_{true} \rvert / L_{true} \times 100\%$ pada setiap sumbu	%
Galat pusat	$\lVert C_{est} - C_{true} \rVert_2$	m
Galat permukaan	Mean jarak titik rekonstruksi ke permukaan referensi terdekat	m
Kelengkapan tampak	Persentase sampel permukaan terlihat yang memiliki tetangga hasil scan pada toleransi δ	%
Pengurangan titik	$(1 - N_{voxel}/N_{raw}) \times 100\%$	%
Laju data valid	N_titik_depth_valid / durasi_simulasi	titik/s

11.1. Desain perbandingan terkontrol

Konfigurasi	Stride	Scan/s	Voxel	Render
Baseline	6	2,22	0,08 m	Pusat sel
Dense only	2	5,00	0,08 m	Pusat sel
Voxel only	6	2,22	0,04 m	Muka sel
Final A+B	2	5,00	0,04 m	Muka sel

Kondisi yang dijaga tetap: lintasan waypoint, pose sensor, objek, pengaturan clipping, dan durasi simulasi. Perbandingan jumlah kandidat sampel 19,04× adalah estimasi teoretis; angka galat dimensi, cakupan dan akurasi permukaan belum tersedia dari berkas hasil scan asli.

12. Rekapitulasi implementasi dan keluaran

Komponen / kegiatan	Status	Keluaran atau bukti
C · Kendali dan jalur	Diterapkan	FlightMission Lua, drone_trajectory.csv, garis biru.
BC · Pemetaan 2D	Diterapkan	01_mapping_2d.png.
B · Scan 3D	Diterapkan	raw_scan.ply, mapping_data.npz.
AB · Voxel postprocess	Diterapkan	voxel_xyz.csv, voxel_height_coloured.ply.
A · Color Z postprocess	Diterapkan	03_voxel_cartesian_3d.png dengan colorbar Z.
Metode C · RANSAC/box	Uji pengembangan	Tidak aktif pada kode final.
Metode D · TSDF	Tidak digunakan	Kajian alternatif.
Akurasi vs ground truth	Belum dievaluasi	Butuh hasil scan scene dan dimensi Cuboid.

12.1. Perintah menjalankan simulasi dan replay

```
python main.py
python main.py --view results/mapping_data.npz
python main.py --view results/mapping_data.npz --mode points
python main.py --view results/mapping_data.npz --headless
```

Skenario replay membaca data NPZ tersimpan tanpa membuka ulang CoppeliaSim. Hasil ekspor `04_display_voxel_cubes.ply` dapat menggunakan sel yang lebih besar jika batas jumlah sel tampilan terlampaui.

12.2. Materi video dan GitHub

Video proses menunjukkan Quadcopter mengikuti waypoint, pose sensor, lintasan, dan pengambilan titik. Video hasil menunjukkan grafik Cartesian yang dapat diputar, lokasi Cuboid terukur, sumbu numerik, serta colorbar Z. Repositori berisi file scene `.ttt`, program `.py`, skrip `.lua`, laporan PDF dan tautan kedua video.

13. Catatan teknis dan referensi

13.1. Batas data dan validitas hasil

Tidak tersedia pasangan data ground truth dan point cloud scene untuk menghitung nilai galat posisi, galat dimensi, atau kelengkapan permukaan aktual. Visualisasi Cartesian yang dicantumkan pada dokumen merupakan hasil pengujian offline sintetis; aset screenshot scene CoppeliaSim merupakan dokumentasi konfigurasi lingkungan. Keterangan tersebut mencegah pencampuran bukti visual sintetis dengan hasil ukur scene.

13.2. Referensi teori dan API

[1] Coppelia Robotics. sim.getVisionSensorDepth — CoppeliaSim API Manual. https://manual.coppeliarobotics.com/en/sim/simGetVisionSensorDepth.htm

[2] Open3D. Point cloud tutorial: Voxel downsampling and Plane segmentation. https://www.open3d.org/docs/latest/tutorial/geometry/pointcloud.html

[3] Fischler, M. A., & Bolles, R. C. (1981). Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography. Communications of the ACM, 24(6), 381–395. https://doi.org/10.1145/358669.358692

[4] Curless, B., & Levoy, M. (1996). A Volumetric Method for Building Complex Models from Range Images. SIGGRAPH. https://www.graphics.stanford.edu/papers/volrange/paper_2_levels/paper.html

[5] Hornung, A., Wurm, K. M., Bennewitz, M., Stachniss, C., & Burgard, W. (2013). OctoMap: An Efficient Probabilistic 3D Mapping Framework Based on Octrees. Autonomous Robots, 34, 189–206. https://doi.org/10.1007/s10514-012-9321-0

13.3. Dasar berkas proyek

`main_uts_fixed.py` menjadi acuan akuisisi awal; `main(2).py` menjadi catatan pengembangan lanjutan; `main_cartesian_clean.py` menjadi implementasi pemetaan akhir dengan 399 baris; `drone_waypoints.lua` memuat logika FlightMission. Perubahan pada skrip Lua tidak diperlukan untuk penambahan visualisasi voxel Cartesian.